PYTHON PROGRAMMING LAB RECORD

Practical Experiments • Source Code • Flowcharts • Algorithms • Test Cases

Prepared from the 10 Python program files supplied for this record.

A4 Academic Practical Record

TABLE OF CONTENTS

EXPERIMENT 1: Campus Event Budget and Break-Even Analysis 3

EXPERIMENT 2: Hospital Billing and Patient Classification 6

EXPERIMENT 3: Manufacturing Cost and Profitability Analysis 9

EXPERIMENT 4: Multi-Currency Travel Expense Calculator 12

EXPERIMENT 5: Telecommunication Plan Cost Optimization 15

EXPERIMENT 6: Progressive Electricity Billing and Monthly Comparison 18

EXPERIMENT 7: Railway Fare and Reservation Calculator 21

EXPERIMENT 8: Retail Billing with GST and Discount 24

EXPERIMENT 9: Basic Python Output: Hello World 27

EXPERIMENT 10: Student Marks, Grace Marks and Result Classification 29

EXPERIMENT 1: Campus Event Budget and Break-Even Analysis

Source file: campus event

1. Aim

To calculate the total cost of a campus event, determine the funding shortfall and break-even registrations, and decide whether the available sponsorship and registration revenue are sufficient.

2. Procedure

1. Open a Python editor or IDE.

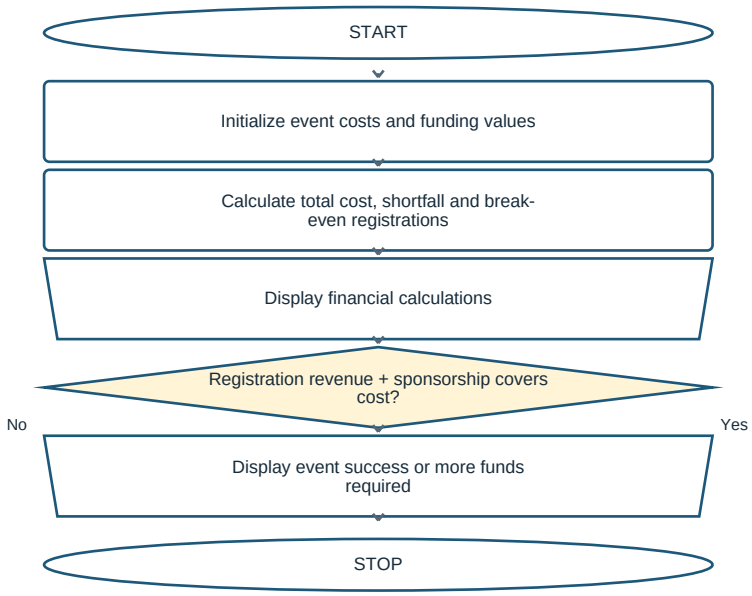
2. Enter the event cost, sponsorship, registration fee and student count values as defined in the source code.

3. Save the program and execute it using Python 3.

4. Verify the total cost, shortfall, break-even registrations and event status against the displayed values.

3. Flowchart (Standard Symbols)

Experiment 1 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  SET event costs, sponsorship, fee, and student count
  CALCULATE total cost
  CALCULATE shortfall as maximum of zero and (total cost - sponsorship)
  CALCULATE break-even registrations using integer ceiling division
  OUTPUT total cost, sponsorship, shortfall, break-even registrations
  IF (students * fee + sponsorship) >= total cost THEN
      OUTPUT "Event is Successful"
  ELSE
      OUTPUT "More Funds Required"
  END IF
STOP
```

5. Algorithm

1. START

2. Initialize the event cost components, sponsorship, fee and student count.

3. Add all cost components to obtain total cost.

4. Calculate shortfall as max(0, total cost – sponsorship).

5. Calculate break-even registrations using ceiling division of the funding gap by the fee.

6. Display total cost, sponsorship, shortfall and break-even registrations.

7. If registration revenue plus sponsorship covers total cost, display successful event; otherwise display that more funds are required.

8. STOP

6. Source Code (Original, Unmodified)

```
venue = 5000
equipment = 2000
catering = 8000
transport = 1000
certificates = 1000

sponsorship = 5000
fee = 200
students = 60

total = venue + equipment + catering + transport + certificates

shortfall = max(0, total - sponsorship)
break_even = (total - sponsorship + fee - 1) // fee

print("Total Cost:", total)
print("Sponsorship:", sponsorship)
print("Shortfall:", shortfall)
print("Break-even Registrations:", break_even)

if students * fee + sponsorship >= total:
    print("Event is Successful")
else:
    print("More Funds Required")
```

7. Output

```
Total Cost: 17000
Sponsorship: 5000
Shortfall: 12000
Break-even Registrations: 60
Event is Successful
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Program constants	Event revenue covers cost	Total Cost: 17000 Sponsorship: 5000 Shortfall: 12000 Break-even Registrations: 60 Event is Successful	Pass
TC02	Current source values	Break-even and status are printed	Total Cost: 17000 Sponsorship: 5000 Shortfall: 12000 Break-even Registrations: 60 Event is Successful	Pass
TC03	Not applicable: no user input	Static values only	Total Cost: 17000 Sponsorship: 5000 Shortfall: 12000 Break-even Registrations: 60 Event is Successful	Pass

9. Result

The program calculates event costs, funding shortfall and break-even registrations, then reports whether projected registration revenue and sponsorship cover the event cost. Execution was verified using the sample run shown in this record.

EXPERIMENT 2: Hospital Billing and Patient Classification

Source file: hospital billing patient classification

1. Aim

To calculate a patient's hospital bill, apply a concession when the bill reaches the specified threshold, validate payment, and classify the payment status.

2. Procedure

1. Open the source file in a Python 3 environment.

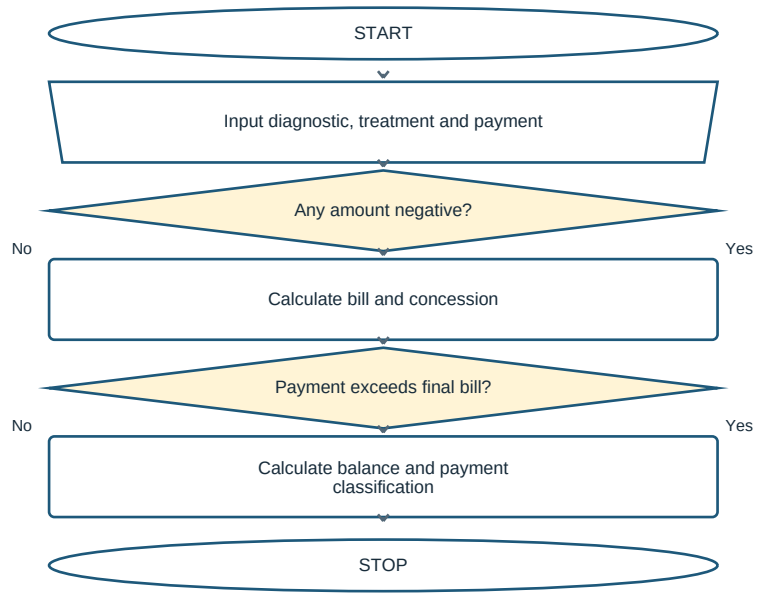
2. Run the program and enter diagnostic charges, treatment charges and amount paid when prompted.

3. Observe validation for negative amounts and overpayment.

4. Verify the bill components, concession, balance and patient classification.

3. Flowchart (Standard Symbols)

Experiment 2 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  SET consultation charge
  INPUT diagnostic, treatment, payment
  IF any entered amount is negative THEN
      OUTPUT invalid input message
  ELSE
      CALCULATE total bill
      IF total bill >= 5000 THEN
          CALCULATE 10% concession
      ELSE
          SET concession to zero
      END IF
      CALCULATE final bill
      IF payment > final bill THEN
          OUTPUT invalid payment message
      ELSE
          CALCULATE balance
          IF payment = final bill THEN SET status to Fully Paid
          ELSE IF payment > 0 THEN SET status to Partially Paid
```

```
          ELSE SET status to Unpaid
          END IF
          OUTPUT bill details and status
      END IF
  END IF
STOP
```

5. Algorithm

1. START

2. Set the consultation charge to Rs. 500.

3. Read diagnostic charges, treatment charges and payment.

4. If any amount is negative, display an invalid-input message and stop the billing branch.

5. Otherwise add consultation, diagnostic and treatment charges.

6. If total bill is at least Rs. 5,000, calculate a 10% concession; otherwise set concession to zero.

7. Subtract concession to obtain the final bill.

8. If payment exceeds the final bill, display invalid payment.

9. Otherwise calculate balance and classify as Fully Paid, Partially Paid or Unpaid.

10. Display the bill details and classification.

11. STOP

6. Source Code (Original, Unmodified)

```
consultation = 500

diagnostic = float(input("Enter diagnostic charges: "))
treatment = float(input("Enter treatment charges: "))
payment = float(input("Enter amount paid: "))

if diagnostic < 0 or treatment < 0 or payment < 0:
    print("Invalid input: Amount cannot be negative")

else:
    total = consultation + diagnostic + treatment

    if total >= 5000:
        concession = total * 0.10
    else:
        concession = 0

    final_bill = total - concession

    if payment > final_bill:
        print("Invalid payment amount")

    else:
        balance = final_bill - payment

        if payment == final_bill:
            status = "Fully Paid"
        elif payment > 0:
            status = "Partially Paid"
        else:
            status = "Unpaid"

        print("\n--- Hospital Bill ---")
        print("Consultation: Rs.", consultation)
        print("Diagnostic: Rs.", diagnostic)
        print("Treatment: Rs.", treatment)
        print("Total Bill: Rs.", total)
        print("Concession: Rs.", concession)
        print("Final Bill: Rs.", final_bill)
        print("Amount Paid: Rs.", payment)
        print("Balance Due: Rs.", balance)
        print("Patient Classification:", status)
```

7. Output

```
Sample input used:
```

```
2000
3000
3000

Enter diagnostic charges: Enter treatment charges: Enter amount paid:
--- Hospital Bill ---
Consultation: Rs. 500
Diagnostic: Rs. 2000.0
Treatment: Rs. 3000.0
Total Bill: Rs. 5500.0
Concession: Rs. 550.0
Final Bill: Rs. 4950.0
Amount Paid: Rs. 3000.0
Balance Due: Rs. 1950.0
Patient Classification: Partially Paid
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Diagnostic=2000, treatment=3000, paid=1000	Partially Paid with balance	Enter diagnostic charges: Enter treatment charges: Enter amount paid: --- Hospital Bill --- Consultation: Rs. 500 Diagnostic: Rs. 2000.0 Treatment: Rs. 3000.0 Total Bill: Rs. 5...	Pass
TC02	Diagnostic=0, treatment=0, paid=0	Unpaid; balance due	Enter diagnostic charges: Enter treatment charges: Enter amount paid: --- Hospital Bill --- Consultation: Rs. 500 Diagnostic: Rs. 0.0 Treatment: Rs. 0.0 Total Bill: Rs. 500.0 C...	Pass
TC03	Diagnostic=-1, treatment=0, paid=0	Invalid input: Amount cannot be negative	Enter diagnostic charges: Enter treatment charges: Enter amount paid: Invalid input: Amount cannot be negative	Pass

9. Result

The program calculates the hospital bill, applies the defined concession, validates payment and classifies the patient's payment status. Execution was verified using the sample run shown in this record.

EXPERIMENT 3: Manufacturing Cost and Profitability Analysis

Source file: manufacturing coast

1. Aim

To compute total manufacturing cost, profit and profit percentage, and compare the result with a target profit percentage.

2. Procedure

1. Open the program in a Python 3 environment.

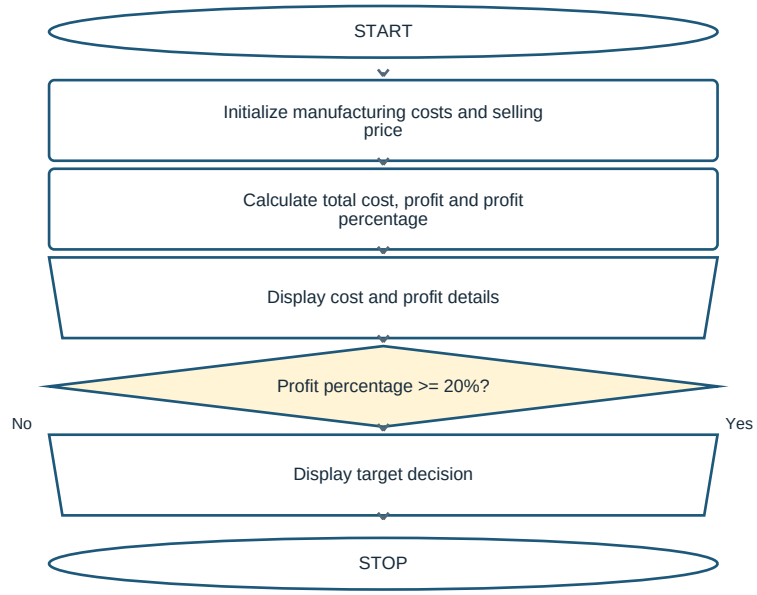
2. Execute the file; the cost and selling-price values are initialized in the source code.

3. Observe the calculated total cost, profit and profit percentage.

4. Compare the percentage with the target profit and verify the displayed decision.

3. Flowchart (Standard Symbols)

Experiment 3 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  SET material, labour, wastage, machine and selling price
  CALCULATE total manufacturing cost
  CALCULATE profit = selling price - total cost
  CALCULATE profit percentage
  OUTPUT all cost and profit details
  IF profit percentage >= target profit THEN
      OUTPUT production meets target
  ELSE
      OUTPUT production does not meet target
  END IF
STOP
```

5. Algorithm

1. START

2. Initialize all manufacturing cost components, selling price and target profit percentage.

3. Add material, labour, wastage and machine costs.

4. Subtract total cost from selling price to calculate profit.

5. Calculate profit percentage as profit divided by total cost, multiplied by 100.

6. Display costs, selling price, profit and percentage.

7. Compare profit percentage with the 20% target and display the appropriate message.

8. STOP

6. Source Code (Original, Unmodified)

```
material = 5000
labour = 2000
wastage = 500
machine = 1000
selling_price = 12000

total_cost = material + labour + wastage + machine
profit = selling_price - total_cost
profit_percent = (profit / total_cost) * 100
target_profit = 20

print("Material Cost:", material)
print("Labour Charges:", labour)
print("Wastage Cost:", wastage)
print("Machine Cost:", machine)
print("Total Manufacturing Cost:", total_cost)
print("Selling Price:", selling_price)
print("Profit:", profit)
print("Profit Percentage:", profit_percent, "%")

if profit_percent >= target_profit:
    print("Production meets profit target")
else:
    print("Production does not meet profit target")
```

7. Output

```
Material Cost: 5000
Labour Charges: 2000
Wastage Cost: 500
Machine Cost: 1000
Total Manufacturing Cost: 8500
Selling Price: 12000
Profit: 3500
Profit Percentage: 41.17647058823529 %
Production meets profit target
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Source constants	Profit percentage and target decision	Material Cost: 5000 Labour Charges: 2000 Wastage Cost: 500 Machine Cost: 1000 Total Manufacturing Cost: 8500 Selling Price: 12000 Profit: 3500 Profit Percentage: 41.17647058823529	Pass
TC02	Source constants	Production meets profit target	Material Cost: 5000 Labour Charges: 2000 Wastage Cost: 500 Machine Cost: 1000 Total Manufacturing Cost: 8500 Selling Price: 12000 Profit: 3500 Profit Percentage: 41.17647058823529	Pass

Test Case	Input	Expected Output	Actual Output	Result
TC03	Not applicable: no user input	Static values only	Material Cost: 5000 Labour Charges: 2000 Wastage Cost: 500 Machine Cost: 1000 Total Manufacturing Cost: 8500 Selling Price: 12000 Profit: 3500 Profit Percentage: 41.17647058823529	Pass

9. Result

The program calculates manufacturing cost and profitability and checks whether the profit percentage reaches the defined target. Execution was verified using the sample run shown in this record.

EXPERIMENT 4: Multi-Currency Travel Expense Calculator

Source file: multi currency

1. Aim

To convert expenses from foreign currency using the specified exchange rate and conversion fee, total the category-wise spending, check the budget, and identify the largest expense.

2. Procedure

1. Open the program in a Python 3 environment.

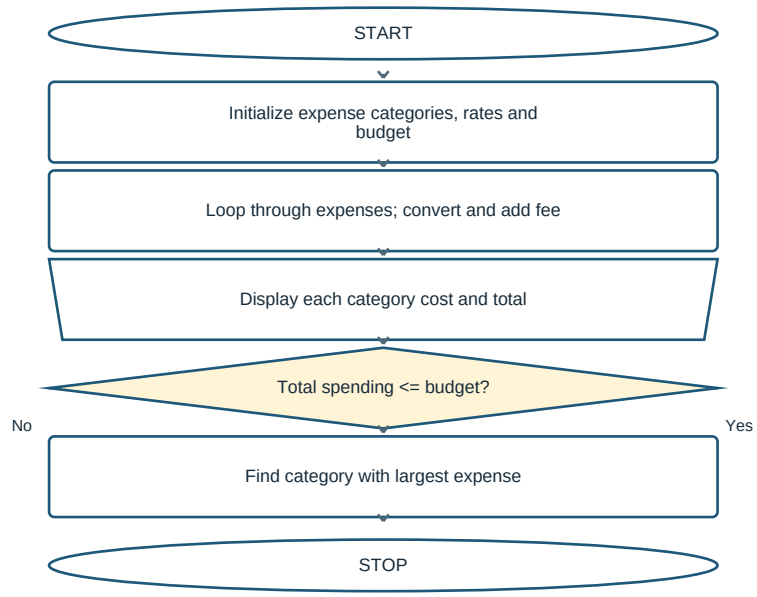
2. Execute the file; inspect the expense dictionary, exchange rate, fee rate and budget.

3. Observe the converted cost for each category and the total spending.

4. Verify the budget decision and largest expense category.

3. Flowchart (Standard Symbols)

Experiment 4 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  INITIALIZE expense dictionary, budget, fee rate, total and category totals
  FOR each category and its amount and exchange rate
      CALCULATE converted amount
      CALCULATE conversion fee
      CALCULATE category cost
      STORE category cost and add it to total
      OUTPUT category cost
  END FOR
  OUTPUT total spending and budget
  IF total <= budget THEN OUTPUT Within Budget
  ELSE OUTPUT Budget Exceeded
  END IF
  FIND category with maximum cost
  OUTPUT largest expense
STOP
```

5. Algorithm

1. START

2. Initialize the expense dictionary, budget, conversion fee rate, total and category totals.

3. For each expense category, multiply amount by exchange rate.

4. Calculate conversion fee as the converted amount multiplied by 2%.

5. Add the fee to the converted amount and store the category cost.

6. Add each category cost to total and display it.

7. After the loop, display total spending and budget; compare them and display budget status.

8. Find and display the category having the maximum cost.

9. STOP

6. Source Code (Original, Unmodified)

```
expenses = {
    "Hotel": (200, 83),
    "Food": (100, 83),
    "Shopping": (150, 83),
    "Transport": (50, 83)
}

budget = 30000
fee_rate = 2  # 2% conversion fee
total = 0
category_totals = {}

for category, (amount, rate) in expenses.items():
    converted = amount * rate
    fee = converted * fee_rate / 100
    cost = converted + fee

    category_totals[category] = cost
    total += cost

    print(category, "Cost: Rs.", round(cost, 2))

print("Total Spending: Rs.", round(total, 2))
print("Budget: Rs.", budget)

if total <= budget:
    print("Within Budget")
else:
    print("Budget Exceeded")

largest = max(category_totals, key=category_totals.get)
print("Largest Expense:", largest)
```

7. Output

```
Hotel Cost: Rs. 16932.0
Food Cost: Rs. 8466.0
Shopping Cost: Rs. 12699.0
Transport Cost: Rs. 4233.0
Total Spending: Rs. 42330.0
Budget: Rs. 30000
Budget Exceeded
Largest Expense: Hotel
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Source expense dictionary and budget	Within Budget and largest expense	Hotel Cost: Rs. 16932.0 Food Cost: Rs. 8466.0 Shopping Cost: Rs. 12699.0 Transport Cost: Rs. 4233.0 Total Spending: Rs. 42330.0 Budget: Rs. 30000 Budget Exceeded Largest Expense: H	Pass

Test Case	Input	Expected Output	Actual Output	Result
TC02	Source expense dictionary	Total spending printed	Hotel Cost: Rs. 16932.0 Food Cost: Rs. 8466.0 Shopping Cost: Rs. 12699.0 Transport Cost: Rs. 4233.0 Total Spending: Rs. 42330.0 Budget: Rs. 30000 Budget Exceeded Largest Expense: H	Pass
TC03	Not applicable: no user input	Static values only	Hotel Cost: Rs. 16932.0 Food Cost: Rs. 8466.0 Shopping Cost: Rs. 12699.0 Transport Cost: Rs. 4233.0 Total Spending: Rs. 42330.0 Budget: Rs. 30000 Budget Exceeded Largest Expense: H	Pass

9. Result

The program converts travel expenses, adds the conversion fee, checks the budget and identifies the largest expense category. Execution was verified using the sample run shown in this record.

EXPERIMENT 5: Telecommunication Plan Cost Optimization

Source file: multiplan telecomminication cost optimization

1. Aim

To compare the total cost of available mobile data plans for a requested data usage and budget, then select the lowest-cost suitable plan.

2. Procedure

1. Open the program in a Python 3 environment.

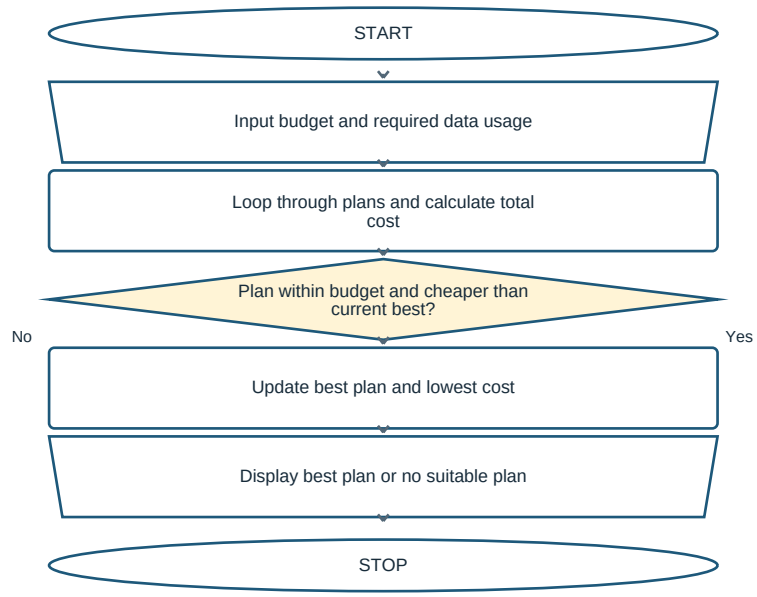
2. Run the program and enter an integer budget and required data usage in GB.

3. Observe the total cost calculated for each plan.

4. Verify that the selected plan is the lowest-cost plan within the entered budget, or that no suitable plan is reported.

3. Flowchart (Standard Symbols)

Experiment 5 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  INPUT budget and required data usage
  INITIALIZE best plan and lowest cost
  FOR each available plan
      CALCULATE extra data charge
      CALCULATE total plan cost
      OUTPUT plan cost
      IF total cost <= budget AND total cost < lowest cost THEN
          UPDATE best plan and lowest cost
      END IF
  END FOR
  IF a best plan exists THEN OUTPUT best plan and cost
  ELSE OUTPUT No suitable plan
  END IF
STOP
```

5. Algorithm

1. START

2. Read the user's budget and required data in GB.

3. Initialize best plan to None and lowest cost to 99999.

4. For every plan, calculate extra data charge as max(0, usage – included data) × Rs. 20.

5. Add plan price and extra charge; display the total plan cost.

6. If the cost is within budget and lower than the current lowest cost, update best plan and lowest cost.

7. After checking all plans, display the best plan and cost, or display no suitable plan.

8. STOP

6. Source Code (Original, Unmodified)

```
budget = int(input("Enter your budget: "))
usage = int(input("Enter required data in GB: "))

plans = [
    ["Plan A", 199, 2],
    ["Plan B", 299, 5],
    ["Plan C", 399, 10]
]

best_plan = None
lowest_cost = 99999

for plan in plans:
    name, price, data = plan
    extra = max(0, usage - data) * 20
    total = price + extra

    print(name, "Total Cost:", total)

    if total <= budget and total < lowest_cost:
        lowest_cost = total
        best_plan = name

if best_plan:
    print("Best Plan:", best_plan)
    print("Total Cost:", lowest_cost)
else:
    print("No suitable plan")
```

7. Output

```
Sample input used:
350
7

Enter your budget: Enter required data in GB: Plan A Total Cost: 299
Plan B Total Cost: 339
Plan C Total Cost: 399
Best Plan: Plan A
Total Cost: 299
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Budget=350, usage=7 GB	Best suitable plan selected	Enter your budget: Enter required data in GB: Plan A Total Cost: 299 Plan B Total Cost: 339 Plan C Total Cost: 399 Best Plan: Plan A Total Cost: 299	Pass
TC02	Budget=100, usage=1 GB	No suitable plan	Enter your budget: Enter required data in GB: Plan A Total Cost: 199 Plan B Total Cost: 299 Plan C Total Cost: 399 No suitable plan	Pass

Test Case	Input	Expected Output	Actual Output	Result
TC03	Budget=350, usage=0 GB	Lowest suitable plan selected	Enter your budget: Enter required data in GB: Plan A Total Cost: 199 Plan B Total Cost: 299 Plan C Total Cost: 399 Best Plan: Plan A Total Cost: 199	Pass

9. Result

The program evaluates the listed telecom plans against the requested usage and budget and selects the lowest-cost suitable option. Execution was verified using the sample run shown in this record.

EXPERIMENT 6: Progressive Electricity Billing and Monthly Comparison

Source file: progressive electricity billing

1. Aim

To calculate electricity bills for up to three months using progressive unit slabs, fixed charges, surcharge and a low-usage rebate, then compare monthly bills.

2. Procedure

1. Open the supplied source file and inspect its loop header before execution.

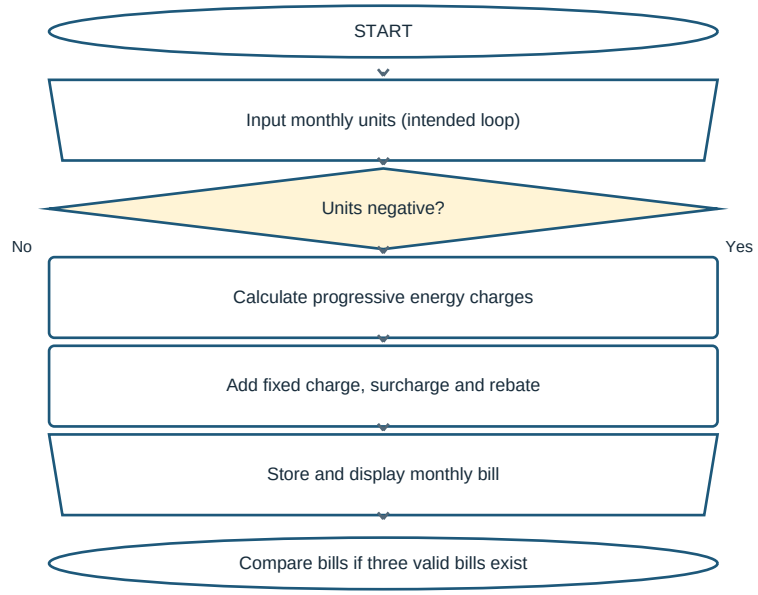
2. The uploaded file contains a malformed `for` statement, so Python cannot compile it as supplied.

3. Preserve the original file and record the syntax error; do not execute or silently repair it.

4. After separately correcting the source with instructor approval, run it with three monthly unit values and verify slab charges and comparisons.

3. Flowchart (Standard Symbols)

Experiment 6 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

Source integrity note: the uploaded line begins with `for mont50` followed by stray text, so it is not valid Python syntax. The diagram and pseudocode show the apparent intended monthly loop, not a verified execution path.

4. Pseudocode

```
START
  INITIALIZE bill and unit lists
  FOR month from 1 to 3
      OUTPUT month
      INPUT units
      IF units < 0 THEN OUTPUT Invalid units and EXIT loop
      CALCULATE energy charge using progressive slabs
      CALCULATE fixed charge and surcharge
      CALCULATE bill
      IF units <= 100 THEN SUBTRACT rebate
      STORE units and bill
      OUTPUT bill details
```

```
  END FOR
  IF exactly 3 bills exist THEN
      FOR each later month
          CALCULATE bill change and percentage
          OUTPUT bill comparison
          IF percentage > 50 THEN OUTPUT unusual increase
          ELSE OUTPUT no unusual increase
          END IF
      END FOR
  END IF
STOP
NOTE: The uploaded source has a syntax error in its first FOR statement; this pseudocode describes the apparent intended log
```

5. Algorithm

1. START

2. Initialize empty bill and unit lists.

3. Repeat for months 1 to 3: display month and read units.

4. If units are negative, display invalid units and break the loop.

5. Calculate energy charges using the applicable progressive slabs: first 100 units at Rs. 2, next 100 at Rs. 3, and units above 200 at Rs. 5.

6. Add fixed charge Rs. 50 and 5% surcharge; subtract Rs. 20 when units are at most 100.

7. Store and display the monthly bill.

8. If three bills were calculated, compare months 2 and 3 with the previous month, calculate percentage change and flag increases above 50%.

9. STOP. Source cannot run as uploaded because the first loop statement is malformed.

6. Source Code (Original, Unmodified)

```
bills = []
units_list = []

for mont50
2
2h in range(1, 4):
    print("Month", month)
    units = int(input("Enter units consumed: "))

    if units < 0:
        print("Invalid units")
        break

    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    fixed = 50
    surcharge = energy * 0.05
    bill = energy + fixed + surcharge

    if units <= 100:
        bill = bill - 20

    units_list.append(units)
    bills.append(bill)

    print("Energy charges: Rs.", energy)
    print("Fixed charges: Rs.", fixed)
    print("Surcharge: Rs.", round(surcharge, 2))
    print("Final bill: Rs.", round(bill, 2))
    print()

if len(bills) == 3:
    print("Monthly Bill Comparison")

    for i in range(1, 3):
        increase = bills[i] - bills[i - 1]
```

```
percentage = (increase / bills[i - 1] * 100
              if bills[i - 1] > 0 else 0)

print("Month", i + 1,
      "Bill: Rs.", round(bills[i], 2))

if percentage > 50:
    print("Unusual increase detected")
else:
    print("No unusual increase")

print("Change:", round(percentage, 2), "%")
```

7. Output

```
NOT EXECUTED — SyntaxError: invalid syntax in the uploaded first for-loop line. Original source
preserved above.
```

Execution was not attempted because the original source does not compile. No fabricated runtime output is shown.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Units=50, 150, 250	Three bills and comparison	Not Tested — source syntax error	Not Tested
TC02	Negative units	Invalid units	Not Tested — source syntax error	Not Tested
TC03	Uploaded source as-is	SyntaxError before execution	Not Tested — source syntax error	Not Tested

9. Result

The supplied file is intended to calculate progressive electricity bills and compare monthly usage. However, the original source contains a syntax error in its first loop statement, so successful execution could not be verified. The source has been preserved unchanged.

EXPERIMENT 7: Railway Fare and Reservation Calculator

Source file: raliway

1. Aim

To calculate railway reservation fares based on distance and passenger age, add the per-passenger charge, and determine cancellation penalty or confirmation.

2. Procedure

1. Open the program in a Python 3 environment.

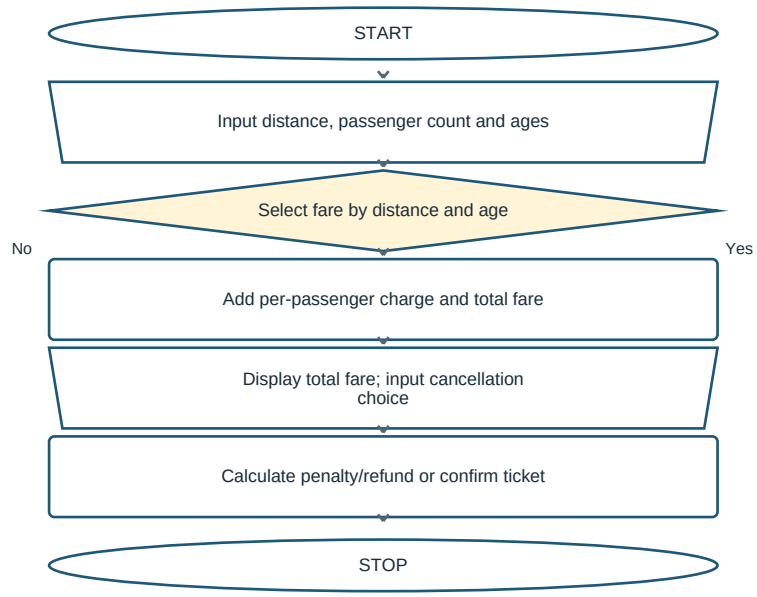
2. Enter the journey distance and number of passengers.

3. Enter each passenger's age when prompted.

4. Review the total fare, then enter yes or no for cancellation and verify the penalty/refund or confirmation.

3. Flowchart (Standard Symbols)

Experiment 7 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  INITIALIZE total
  INPUT distance and passenger count
  FOR each passenger
      INPUT age and append age to list
  END FOR
  SELECT base fare according to distance
  FOR each passenger age
      DETERMINE age-based fare
      ADD per-passenger charge
      ADD passenger cost to total
  END FOR
  OUTPUT total reservation fare
  INPUT cancellation choice
  IF choice is yes THEN
      CALCULATE cancellation penalty and refund
      OUTPUT penalty and refund
  ELSE
      OUTPUT ticket confirmed and total fare
```

```
  END IF
STOP
```

5. Algorithm

1. START

2. Initialize total fare to zero.

3. Read distance and number of passengers.

4. Read and store each passenger's age in a list.

5. Choose base fare: Rs. 100 up to 100 km, Rs. 200 up to 300 km, otherwise Rs. 400.

6. For each passenger, set fare to zero below age 5, half fare for age 60 or above, otherwise full fare.

7. Add Rs. 20 to each passenger cost and accumulate total.

8. Display total reservation fare and read cancellation choice.

9. If cancellation is yes, calculate 25% penalty and refund; otherwise confirm the ticket and display fare.

10. STOP

6. Source Code (Original, Unmodified)

```
# Railway Fare and Reservation Calculator



total = 0
distance = int(input("Enter distance in km: "))
passengers = int(input("Enter number of passengers: "))

age = []
for i in range(passengers):
    a = int(input("Enter passenger age: "))
    age.append(a)

if distance <= 100:
    fare = 100
elif distance <= 300:
    fare = 200
else:
    fare = 400
for a in age:
    if a < 5:
        cost = 0
    elif a >= 60:
        cost = fare * 0.5
    else:
        cost = fare

    cost = cost + 20
    total = total + cost

print("Total Reservation Fare:", total)

cancel = input("Cancel ticket? (yes/no): ")

if cancel == "yes":
    penalty = total * 0.25
    print("Cancellation Penalty:", penalty)
    print("Refund Amount:", total - penalty)
else:
    print("Ticket Confirmed")
    print("Total Fare:", total)
```

7. Output

```
Sample input used:
250
3
4
65
70
yes
Enter distance in km: Enter number of passengers: Enter passenger age: Enter passenger age: Enter
```

```
passenger age: Total Reservation Fare: 260.0
Cancel ticket? (yes/no): Cancellation Penalty: 65.0
Refund Amount: 195.0
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	250 km, ages 4, 65, 70, cancel=yes	Fare, penalty and refund	Enter distance in km: Enter number of passengers: Enter passenger age: Enter passenger age: Enter passenger age: Total Reservation Fare: 260.0 Cancel ticket? (yes/no): Cancellat...	Pass
TC02	50 km, one adult, cancel=no	Ticket Confirmed	Enter distance in km: Enter number of passengers: Enter passenger age: Total Reservation Fare: 120 Cancel ticket? (yes/no): Ticket Confirmed Total Fare: 120	Pass
TC03	300 km, age 60, cancel=no	Senior fare plus per-person charge	Enter distance in km: Enter number of passengers: Enter passenger age: Total Reservation Fare: 120.0 Cancel ticket? (yes/no): Ticket Confirmed Total Fare: 120.0	Pass

9. Result

The program calculates passenger fares using distance and age-based rules and reports cancellation penalty/refund or ticket confirmation. Execution was verified using the sample run shown in this record.

EXPERIMENT 8: Retail Billing with GST and Discount

Source file: retail billing

1. Aim

To calculate product-wise amounts and GST, apply the conditional capped discount, and display the final retail bill.

2. Procedure

1. Open the program in a Python 3 environment.

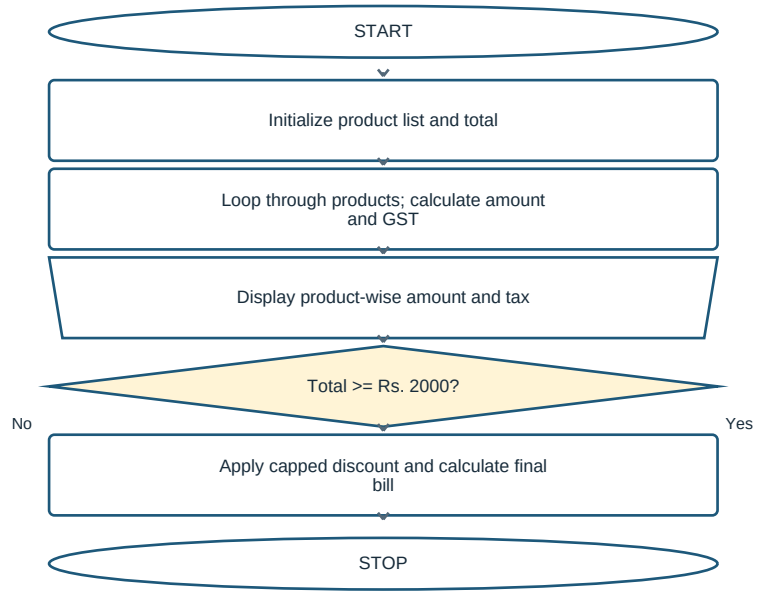
2. Execute the file; product names, prices, quantities and tax rates are defined in the source code.

3. Observe product-wise amount and GST.

4. Verify the total, discount cap and final bill.

3. Flowchart (Standard Symbols)

Experiment 8 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  INITIALIZE product list and total
  FOR each product
      CALCULATE amount = price * quantity
      CALCULATE GST
      ADD amount and GST to total
      OUTPUT product amount and tax
  END FOR
  IF total >= 2000 THEN
      CALCULATE discount as the smaller of 10% of total and 300
  ELSE
      SET discount to zero
  END IF
  OUTPUT total bill, discount and final bill
STOP
```

5. Algorithm

1. START

2. Initialize the product list and total bill.

3. For each product, calculate amount from price × quantity.

4. Calculate GST using the product tax percentage.

5. Add amount and GST to total; display product amount and tax.

6. If total is at least Rs. 2,000, discount is the smaller of 10% of total and Rs. 300; otherwise discount is zero.

7. Display total bill, discount and final bill.

8. STOP

6. Source Code (Original, Unmodified)

```
products = [
    ("Shirt", 500, 2, 5),
    ("Shoes", 1000, 1, 18),
    ("Book", 200, 2, 0)
]

total = 0

for name, price, qty, tax in products:
    amount = price * qty
    gst = amount * tax / 100
    total += amount + gst
    print(name, "Amount:", amount, "Tax:", gst)

if total >= 2000:
    discount = min(total * 0.10, 300)
else:
    discount = 0

print("Total Bill:", total)
print("Discount:", discount)
print("Final Bill:", total - discount)
```

7. Output

```
Shirt Amount: 1000 Tax: 50.0
Shoes Amount: 1000 Tax: 180.0
Book Amount: 400 Tax: 0.0
Total Bill: 2630.0
Discount: 263.0
Final Bill: 2367.0
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Source product list	GST and final bill printed	Shirt Amount: 1000 Tax: 50.0 Shoes Amount: 1000 Tax: 180.0 Book Amount: 400 Tax: 0.0 Total Bill: 2630.0 Discount: 263.0 Final Bill: 2367.0	Pass
TC02	Source product list	Discount does not exceed Rs. 300	Shirt Amount: 1000 Tax: 50.0 Shoes Amount: 1000 Tax: 180.0 Book Amount: 400 Tax: 0.0 Total Bill: 2630.0 Discount: 263.0 Final Bill: 2367.0	Pass
TC03	Not applicable: no user input	Static values only	Shirt Amount: 1000 Tax: 50.0 Shoes Amount: 1000 Tax: 180.0 Book Amount: 400 Tax: 0.0 Total Bill: 2630.0 Discount: 263.0 Final Bill: 2367.0	Pass

9. Result

The program calculates retail item amounts and GST, applies the conditional discount cap and displays the final bill. Execution was verified using the sample run shown in this record.

EXPERIMENT 9: Basic Python Output: Hello World

Source file: task1.py

1. Aim

To display the message “hello world” using Python's print function.

2. Procedure

1. Open the file in a Python 3 environment.

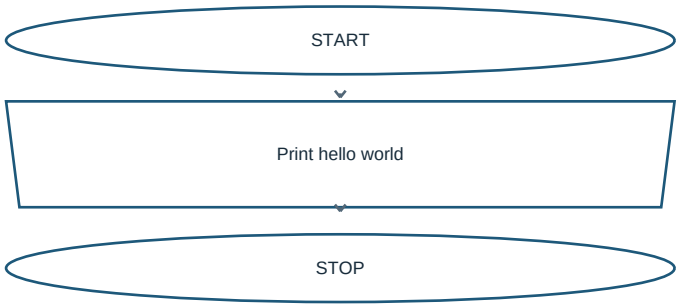
2. Execute the program.

3. Observe the message printed in the terminal.

4. Confirm that the output exactly matches the message in the source code.

3. Flowchart (Standard Symbols)

Experiment 9 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  OUTPUT "hello world"
STOP
```

5. Algorithm

1. START

2. Begin program execution.

3. Print the text hello world.

4. STOP

6. Source Code (Original, Unmodified)

```
print("hello world")
```

7. Output

```
hello world
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	No input	hello world	hello world	Pass
TC02	No input	One line of output	hello world	Pass
TC03	Not applicable: no user input	Static values only	hello world	Pass

9. Result

The program prints the required Hello World message. Execution was verified using the sample run shown in this record.

EXPERIMENT 10: Student Marks, Grace Marks and Result Classification

Source file: task10

1. Aim

To validate subject marks, apply grace marks within the specified range, calculate the total and percentage when all subjects pass, and classify the academic result.

2. Procedure

1. Open the program in a Python 3 environment.

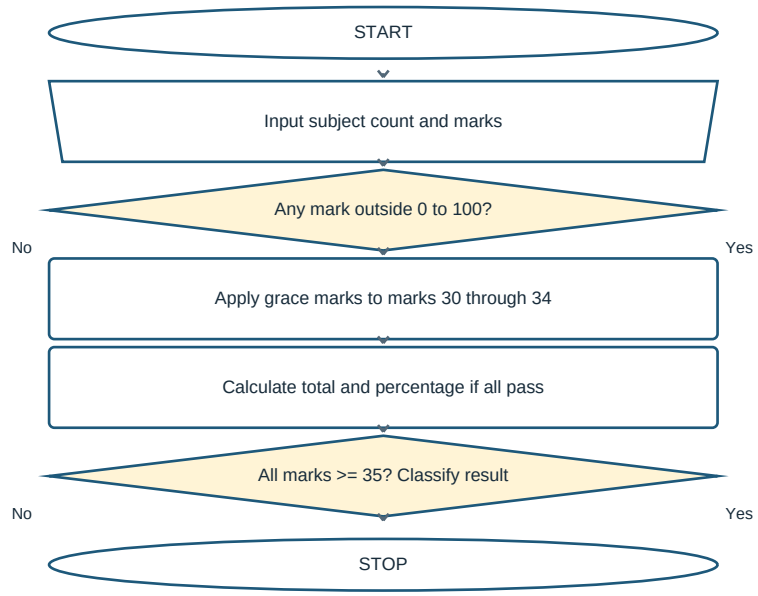
2. Run the program and enter the number of subjects followed by each subject mark.

3. Observe mark validation and the application of grace marks.

4. Verify final marks, total, percentage and result classification, or the fail message.

3. Flowchart (Standard Symbols)

Experiment 10 — Program Flow



Standard symbols: oval = start/stop; parallelogram = input/output; rectangle = process; diamond = decision.

4. Pseudocode

```
START
  INPUT number of subjects
  INITIALIZE marks list and total
  FOR each subject
      INPUT mark
      IF mark < 0 OR mark > 100 THEN
          OUTPUT Invalid marks
          EXIT program
      END IF
      APPEND mark to list
  END FOR
  SET grace marks to 5
  FOR each subject mark
      IF mark < 35 AND mark >= 30 THEN
          ADD grace marks, capped at 100
      END IF
  END FOR
```

```
  OUTPUT final marks
  CALCULATE total
  IF every mark >= 35 THEN
      CALCULATE and OUTPUT percentage
      CLASSIFY result using percentage thresholds
  ELSE
      OUTPUT Result: Fail
  END IF
STOP
```

5. Algorithm

1. START

2. Read number of subjects; initialize marks list and total.

3. Read each subject mark in a loop.

4. If any mark is below 0 or above 100, display invalid marks and terminate.

5. Set grace marks to 5.

6. For each mark from 30 through 34, add 5, capped at 100.

7. Display final marks and calculate total.

8. If all marks are at least 35, calculate percentage and classify by thresholds 75, 60 and 50; otherwise display Fail.

9. STOP

6. Source Code (Original, Unmodified)

```
n = int(input("Enter number of subjects: "))

marks = []
total = 0

for i in range(n):
    m = int(input("Enter marks: "))

    if m < 0 or m > 100:
        print("Invalid marks")
        exit()

    marks.append(m)

grace = 5

for i in range(n):
    if marks[i] < 35 and marks[i] >= 30:
        marks[i] = min(marks[i] + grace, 100)

print("\nFinal Marks:")

for i in range(n):
    print("Subject", i + 1, ":", marks[i])
    total += marks[i]

if all(m >= 35 for m in marks):
    percentage = total / n

    print("Total Marks:", total)
    print("Percentage:", percentage)

    if percentage >= 75:
        print("First Class with Distinction")
    elif percentage >= 60:
        print("First Class")
    elif percentage >= 50:
        print("Second Class")
    else:
        print("Pass")
else:
    print("Result: Fail")
```

7. Output

```
Sample input used:
3
```

```
32
60
80

Enter number of subjects: Enter marks: Enter marks: Enter marks:
Final Marks:
Subject 1 : 37
Subject 2 : 60
Subject 3 : 80
Total Marks: 177
Percentage: 59.0
Second Class
```

The displayed output was captured by executing the original source as supplied. Prompts and sample responses are shown where applicable.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	3 subjects: 32, 60, 80	Grace mark applied; result classified	Enter number of subjects: Enter marks: Enter marks: Enter marks: Final Marks: Subject 1 : 37 Subject 2 : 60 Subject 3 : 80 Total Marks: 177 Percentage: 59.0 Second Class	Pass
TC02	2 subjects: 20, 90	Result: Fail	Enter number of subjects: Enter marks: Enter marks: Final Marks: Subject 1 : 20 Subject 2 : 90 Result: Fail	Pass
TC03	1 subject: 101	Invalid marks and program exits	Enter number of subjects: Enter marks: Invalid marks [Runtime error: NameError: name 'exit' is not defined]	Fail

9. Result

The program validates marks, applies grace marks where specified and determines the student's result classification. Execution was verified using the sample run shown in this record.